

Problem Set II - QPROG

Implementing the Bernstein-Vazirani Protocol in Qiskit

Mateus de Oliveira Oliveira
Department of Computer and System Sciences
Stockholm University

This is a guided implementation assignment. Please deliver your code as a **single** Jupyter notebook file, containing the implementation of each function described below, together with experimental results giving evidence that the code is correct. For example, by printing the circuit corresponding to each section and by testing the Bernstein-Vazirani algorithm with different inputs.

1 Bernstein-Vazirani

Given two bits b_1 and b_2 , we let $b_1 \oplus b_2$ denote the 2-bit XOR of b_1 with b_2 , and $b_1 b_2$ denote the AND of b_1 and b_2 . Given two n -bit strings a and x , we let

$$a \cdot x = a_0 x_0 \oplus a_1 x_1 \oplus \cdots \oplus a_{n-1} x_{n-1}$$

be the inner product between a and x , modulo 2. That is, we take the AND of a and x bit by bit, and then XOR the resulting bits.

In the *Bernstein-Vazirani* problem we are given a function

$$f : \{0, 1\}^n \rightarrow \{0, 1\}$$

with the promise that there is some secret string $a \in \{0, 1\}^n$ such that for each $x \in \{0, 1\}^n$, $f(x) = a \cdot x$. In other words, when given a string x as input, the function f computes the inner product between a and x . The goal is to discover the secret string a by querying f as few times as possible. Here, querying f at a specific entry x means computing the value of f at x , that is, the bit $f(x)$. Here we assume that f is implemented as a black-box circuit. That is, we can interact with the black box by querying it, but we cannot see the details of the implementation.

Classically, this problem can be solved with n queries: for each $i \in \{0, 1, \dots, n-1\}$, to discover the bit a_i , just query the function f on the string x where all bits are 0, except for the i -th bit ($x_i = 1$). Surprisingly, with a quantum circuit one can discover a by querying f a single time.

2 Implementing Inner Products as a Black Box

In order to demonstrate Bernstein-Vazirani's protocol in qiskit, we first need to implement a quantum circuit with n input qubits and one output qubit, which sends each basis state $|x\rangle_n |b\rangle_1$ to the basis state $|x\rangle_n |b \oplus a \cdot x\rangle_1$. Here, the subscripts n and 1 are just indicating the number of qubits in the input register and in the output register respectively.

Interestingly, inner products modulo two can be computed quite easily using CNOT gates. More specifically, if a is an n -bit string, then the circuit computing $a \cdot x$ consists of a sequence of CNOT gates, one for each position i such that $a_i = 1$. In all these cases, the control bit is a_i , while the target bit is the output bit. For example, if $a = 01101$, then the circuit is the following.

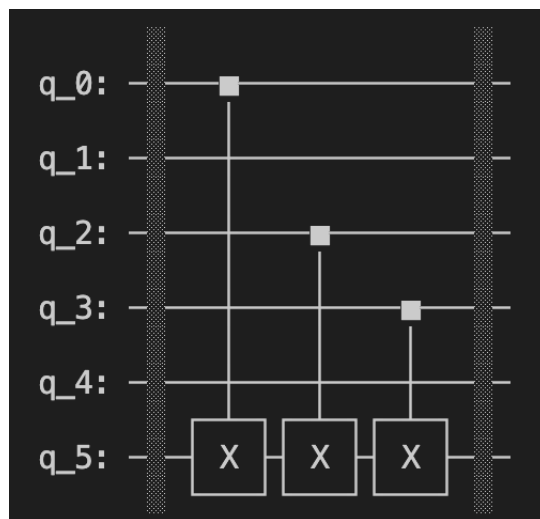


Figure 1: Inner product circuit corresponding to string $a = 01101$. Here, q_0 is the last significant bit of a , while q_4 is the most significant bit of a . The bit q_5 is the output bit. The white squares are the control bits. The squares with an X are the target bits.

Task 1 (Implementing the Inner Product Function). *Implement a function `inner_product(circuit, a)` that takes as input a quantum circuit `circuit` and a binary string `a`, and adds to `circuit` the CNOT gates corresponding to `a`. You can assume that the circuit has $n+1$ qubits, numbered from $0, 1, \dots, n$, where $n = \text{len}(a)$.*

Please pay attention to the following remarks.

Remark 1. The function `inner_product(circuit,a)` does not return a value. The circuit is simply modified by the function.

Remark 2. The qubit q_n is the output qubit, while q_0 is the least significant bit of the input register and q_{n-1} is the most significant bit of the input register. Note that in the string a , a_0 is the most significant bit, while a_{n-1} is the least significant bit. So you should take this into account when adding the CNOT gates. Hint: define the circuit using the reverse of a .

To check that whether your code is correct you can use the code snippet below. If everything is working correctly, the circuit of Figure 1 will be printed when $a = 01101$.

```
a = "01101"
n=len(a)
circuit = QuantumCircuit(n+1,0)
inner_product(circuit,a)
print(circuit)
```

Remark 3. It is advisable to add vertical bars to separate blocks of instructions. This is done with the method `circuit.barrier()`.

3 Adding a layer of Hadamard Gates

The next step is to implement a function that adds a layer of hadamard gates to the qubits of the circuit.

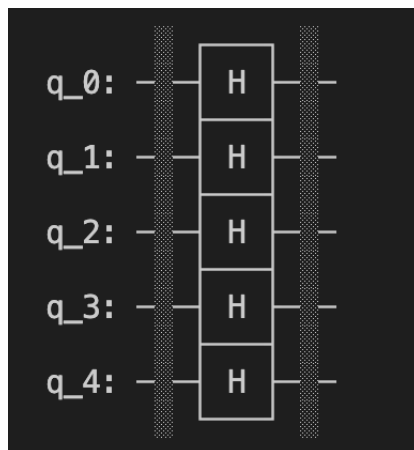


Figure 2: Layer of Hadamard Gates.

Task 2. Implement a function `Hadamards(circuit)` that takes as input a quantum circuit `circuit` and adds one Hadamard gate to each qubit of the circuit. To get the number of qubits, use the attribute `circuit.num_qubits`.

To test whether your function is working correctly, use the following code snippet. If everything is working correctly, the circuit of Figure 2 will be printed.

```
circuit = QuantumCircuit(5,0)
hadamards(circuit)
print(circuit)
```

4 Constructing the Bernstein-Vazirani Circuit

Now that you have the functions `inner_product(circuit,a)` and `Hadamards(circuit)`, the next step is to create a function that constructs the Bernstein-Vazirani Circuit.

Task 3. *Implement a function `bernstein_vazirani(a)` that takes as input a binary string a and returns a quantum circuit implementing the Bernstein-Vazirani protocol with respect to the secret bit string a . Your function performs the following steps:*

1. *Let $n=\text{len}(a)$*
2. *Create a quantum circuit `circuit` with $n + 1$ qubits and n bits.*
3. *Apply the X gate to flip the output qubit from $|0\rangle$ to $|1\rangle$.*
4. *Apply a layer of Hadamard Gates.*
5. *Apply the inner product circuit with respect to a .*
6. *Apply another layer of Hadamard Gates.*
7. *Measure the n input qubits $0, 1, \dots, n - 1$ and store the result into the classical qubits $0, 1, \dots, n - 1$.*
8. *return the circuit.*

To test whether your code is correct, you can use the following code snippet. If everything went well you should see the circuit of Figure 3.

```
a = "01101"
circuit = bernstein_vazirani(a)
print(circuit)
```

Important: The construction should work with an arbitrary string a . In particular, a may have any number of leading zeros. For example, if $a = 0000101$, then the final circuit should have 7 input qubits and one output qubit.

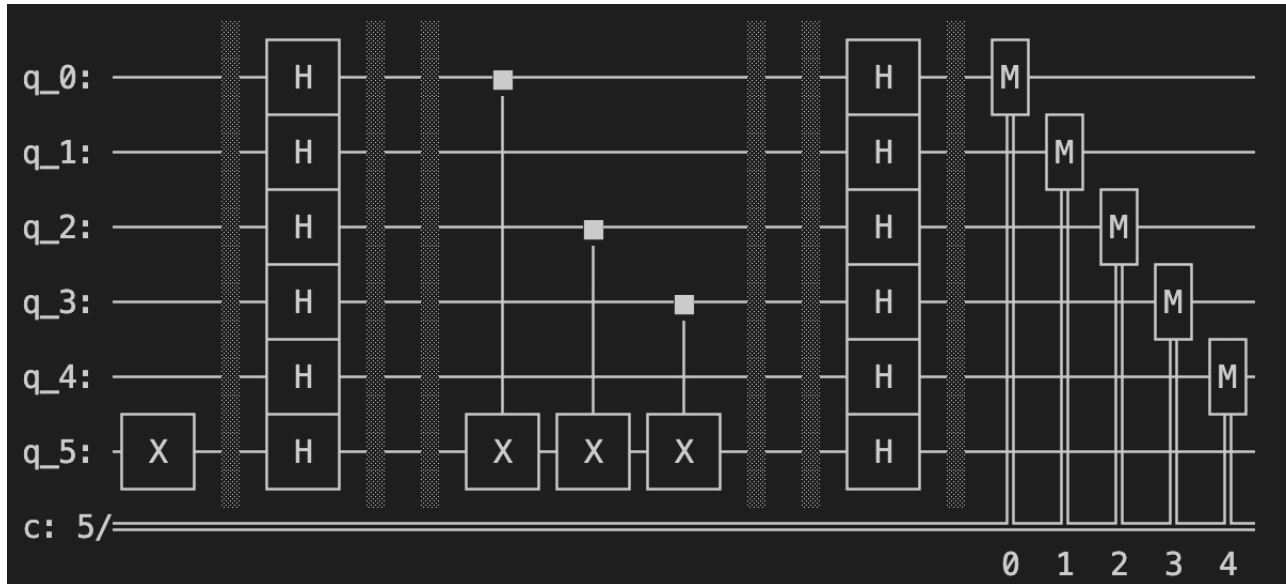


Figure 3: Bernstein-Vazirani Circuit.

5 Simulating the circuit.

Finally, once we have constructed the circuit we should simulate it. Here, you can choose any simulation method you wish. That is, using `BasicSimulator()`, `AerSimulator()`, a simulator on the cloud, or a real quantum computer on the cloud.

Task 4. *Simulate your circuit using a method of your choice. But for the submission, please use either `BasicSimulator()` or `AerSimulator()`, since I need to execute your code locally.*

Important: Please remove any references to eventual API keys from the submission file.

When simulating your code, the output distribution should be concentrated in the basis state corresponding to the string a . For example, when using one of the suggested local simulators, the result should be similar to the one below.

{13: 0.9999999999999998}

Figure 4: Output ditribution.

Here, we have that the state of the system collapses to the basis state 01101 (corresponding to the binary representation of 13) with essentially 100% probability. Note that 01101 is precisely the secret string used to define the function f in our example.

Remark: Please note that when running the code on a real quantum computer through the cloud, some other basis states may also show up with a small probability.